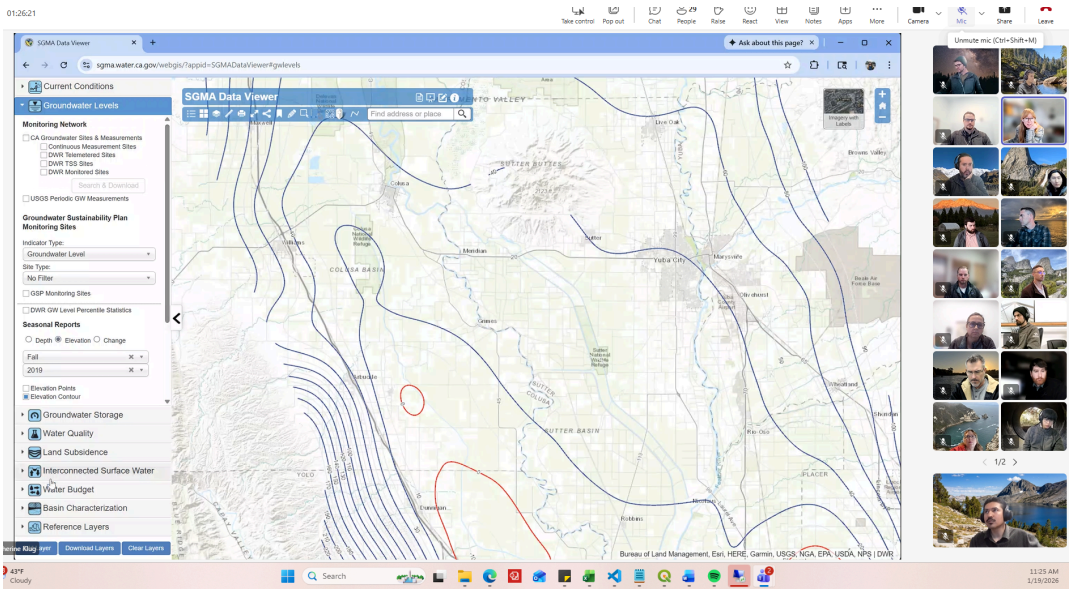
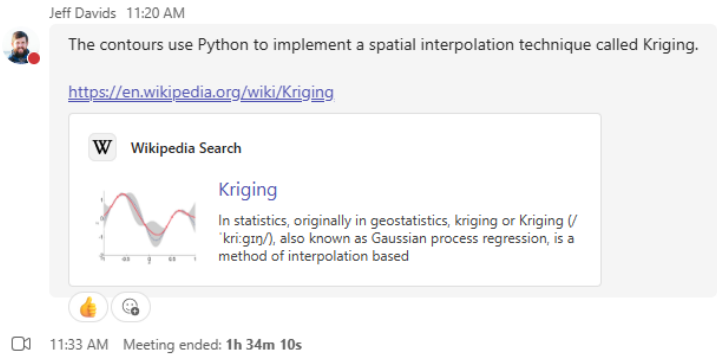
Simple Kriging Variance

DE resources

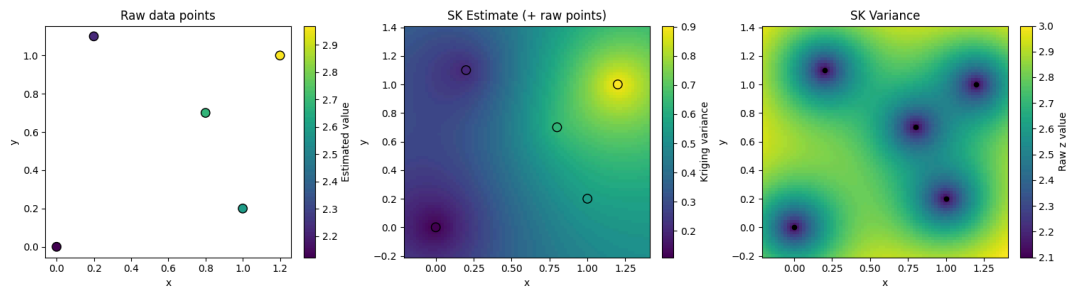
🚿 DE

Inspired by solution specific meeting





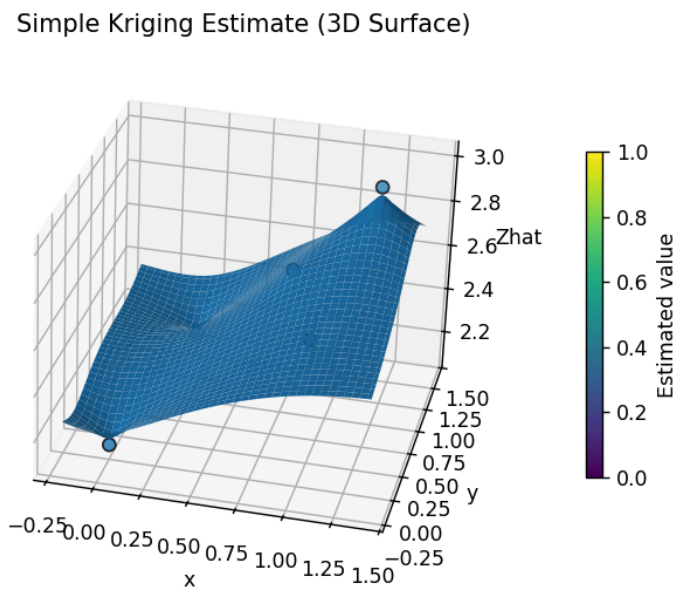
https://en.wikipedia.org/wiki/Kriging




```
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# 1) Example sample data
# ----------------------------
# Coordinates of measurements (x, y) and observed values z
X = np.array([
    [0.0, 0.0],
    [1.0, 0.2],
    [1.2, 1.0],
    [0.2, 1.1],
    [0.8, 0.7],
])
z = np.array([2.1, 2.6, 3.0, 2.2, 2.7])

# Simple kriging assumption: known global mean
mu = 2.5

# ----------------------------
# 2) Covariance model
# ----------------------------
# Exponential covariance:
# C(h) = sill * exp(-h / a)
# Optional nugget handled via diagonal: C(0) += nugget
sill = 1.0
a = 0.6      # "range-like" parameter controlling correlation decay
nugget = 0.05

def exp_cov(h, sill, a):
    return sill * np.exp(-h / a)

def pairwise_dist(A, B):
    # returns matrix of Euclidean distances between points in A and B
    # A: (n,2), B: (m,2) -> (n,m)
    diff = A[:, None, :] - B[None, :, :]
    return np.sqrt(np.sum(diff**2, axis=2))
```

```
# ----------------------------
# 3) Simple kriging function
# ----------------------------
def simple_kriging_predict(X, z, x0, mu, sill, a, nugget):
    """
    Returns (z_hat, sigma2) at location x0 using simple kriging.
    """
    X = np.asarray(X, float)
    z = np.asarray(z, float)
    x0 = np.asarray(x0, float).reshape(1, 2)

    # Covariance among data points
    D = pairwise_dist(X, X)                 # (n,n)
    C = exp_cov(D, sill, a)
    C = C + np.eye(len(X)) * nugget         # nugget on diagonal

    # Covariance between data and target point
    d0 = pairwise_dist(X, x0).reshape(-1)   # (n,)
    c0 = exp_cov(d0, sill, a)               # (n,)

    # Solve for weights: w = C^{-1} c0
    w = np.linalg.solve(C, c0)

    # Simple kriging estimate: mu + w^T (z - mu)
    z_hat = mu + np.dot(w, (z - mu))

    # Kriging variance: C(0) - w^T c0
    # Here C(0) is sill + nugget (variance at a point)
    C00 = sill + nugget
    sigma2 = C00 - np.dot(w, c0)

    return z_hat, sigma2

# ----------------------------
# 4) Predict on a grid
# ----------------------------
grid_x = np.linspace(-0.2, 1.4, 80)
grid_y = np.linspace(-0.2, 1.4, 80)
GX, GY = np.meshgrid(grid_x, grid_y)

Zhat = np.zeros_like(GX)
Var  = np.zeros_like(GX)

for i in range(GX.shape[0]):
    for j in range(GX.shape[1]):
        z_hat, s2 = simple_kriging_predict(
            X, z, [GX[i, j], GY[i, j]], mu, sill, a, nugget
        )
```

```
        Zhat[i, j] = z_hat
        Var[i, j]  = s2


import matplotlib.pyplot as plt

fig, axs = plt.subplots(1, 3, figsize=(15, 4), constrained_layout=True)

# 1) Kriging estimate + raw points
m1 = axs[1].pcolormesh(GX, GY, Zhat, shading="auto")
fig.colorbar(m1, ax=axs[0], label="Estimated value")
axs[1].scatter(X[:,0], X[:,1], c=z, edgecolor="k", s=80)
axs[1].set_title("SK Estimate (+ raw points)")
axs[1].set_xlabel("x"); axs[1].set_ylabel("y")

# 2) Kriging variance
m2 = axs[2].pcolormesh(GX, GY, Var, shading="auto")
fig.colorbar(m2, ax=axs[1], label="Kriging variance")
axs[2].scatter(X[:,0], X[:,1], c="k", s=20)
axs[2].set_title("SK Variance")
axs[2].set_xlabel("x"); axs[2].set_ylabel("y")

# 3) Raw points only
s3 = axs[0].scatter(X[:,0], X[:,1], c=z, edgecolor="k", s=80)
fig.colorbar(s3, ax=axs[2], label="Raw z value")
axs[0].set_title("Raw data points")
axs[0].set_xlabel("x"); axs[0].set_ylabel("y")

plt.show()


# plt.show()

# Example single-point prediction:
x0 = [0.6, 0.3]
pred, s2 = simple_kriging_predict(X, z, x0, mu, sill, a, nugget)
print(f"Prediction at {x0}: {pred:.3f}   (std = {np.sqrt(max(s2,0)):.3f})")
```



version with a 3d surface

```
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# 1) Example sample data
# ----------------------------
# Coordinates of measurements (x, y) and observed values z
X = np.array([
    [0.0, 0.0],
    [1.0, 0.2],
    [1.2, 1.0],
    [0.2, 1.1],
    [0.8, 0.7],
])
z = np.array([2.1, 2.6, 3.0, 2.2, 2.7])

# Simple kriging assumption: known global mean
mu = 2.5

# ----------------------------
# 2) Covariance model
# ----------------------------
# Exponential covariance:
# C(h) = sill * exp(-h / a)
# Optional nugget handled via diagonal: C(0) += nugget
sill = 1.0
```

```
a = 0.6      # "range-like" parameter controlling correlation decay
nugget = 0.05

def exp_cov(h, sill, a):
    return sill * np.exp(-h / a)

def pairwise_dist(A, B):
    # returns matrix of Euclidean distances between points in A and B
    # A: (n,2), B: (m,2) -> (n,m)
    diff = A[:, None, :] - B[None, :, :]
    return np.sqrt(np.sum(diff**2, axis=2))

# ----------------------------
# 3) Simple kriging function
# ----------------------------
def simple_kriging_predict(X, z, x0, mu, sill, a, nugget):
    """
    Returns (z_hat, sigma2) at location x0 using simple kriging.
    """
    X = np.asarray(X, float)
    z = np.asarray(z, float)
    x0 = np.asarray(x0, float).reshape(1, 2)

    # Covariance among data points
    D = pairwise_dist(X, X)                 # (n,n)
    C = exp_cov(D, sill, a)
    C = C + np.eye(len(X)) * nugget         # nugget on diagonal

    # Covariance between data and target point
    d0 = pairwise_dist(X, x0).reshape(-1)   # (n,)
    c0 = exp_cov(d0, sill, a)               # (n,)

    # Solve for weights: w = C^{-1} c0
    w = np.linalg.solve(C, c0)

    # Simple kriging estimate: mu + w^T (z - mu)
    z_hat = mu + np.dot(w, (z - mu))

    # Kriging variance: C(0) - w^T c0
    # Here C(0) is sill + nugget (variance at a point)
    C00 = sill + nugget
    sigma2 = C00 - np.dot(w, c0)

    return z_hat, sigma2

# ----------------------------
# 4) Predict on a grid
# ----------------------------
```

```
grid_x = np.linspace(-0.2, 1.4, 80)
grid_y = np.linspace(-0.2, 1.4, 80)
GX, GY = np.meshgrid(grid_x, grid_y)

Zhat = np.zeros_like(GX)
Var  = np.zeros_like(GX)

for i in range(GX.shape[0]):
    for j in range(GX.shape[1]):
        z_hat, s2 = simple_kriging_predict(
            X, z, [GX[i, j], GY[i, j]], mu, sill, a, nugget
        )
        Zhat[i, j] = z_hat
        Var[i, j]  = s2


import matplotlib.pyplot as plt

fig, axs = plt.subplots(1, 3, figsize=(15, 4), constrained_layout=True)

# 1) Kriging estimate + raw points
m1 = axs[1].pcolormesh(GX, GY, Zhat, shading="auto")
fig.colorbar(m1, ax=axs[0], label="Estimated value")
axs[1].scatter(X[:,0], X[:,1], c=z, edgecolor="k", s=80)
axs[1].set_title("SK Estimate (+ raw points)")
axs[1].set_xlabel("x"); axs[1].set_ylabel("y")

# 2) Kriging variance
m2 = axs[2].pcolormesh(GX, GY, Var, shading="auto")
fig.colorbar(m2, ax=axs[1], label="Kriging variance")
axs[2].scatter(X[:,0], X[:,1], c="k", s=20)
axs[2].set_title("SK Variance")
axs[2].set_xlabel("x"); axs[2].set_ylabel("y")

# 3) Raw points only
s3 = axs[0].scatter(X[:,0], X[:,1], c=z, edgecolor="k", s=80)
fig.colorbar(s3, ax=axs[2], label="Raw z value")
axs[0].set_title("Raw data points")
axs[0].set_xlabel("x"); axs[0].set_ylabel("y")




import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401

# --- 3D surface: Simple Kriging estimate ---
fig = plt.figure()
```

```
ax = fig.add_subplot(111, projection="3d")
surf = ax.plot_surface(GX, GY, Zhat, linewidth=0, antialiased=True)

ax.set_title("Simple Kriging Estimate (3D Surface)")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("Zhat")

fig.colorbar(surf, ax=ax, shrink=0.6, pad=0.1, label="Estimated value")

# Optional: show raw points as 3D scatter
ax.scatter(X[:,0], X[:,1], z, edgecolor="k", s=40)

plt.show()

# plt.show()


# plt.show()

# Example single-point prediction:
x0 = [0.6, 0.3]
pred, s2 = simple_kriging_predict(X, z, x0, mu, sill, a, nugget)
print(f"Prediction at {x0}: {pred:.3f}   (std = {np.sqrt(max(s2,0)):.3f})")
```